

Upgrade Path

GUIDE OPÉRATIONNEL

Portainer Repository • GHCR • HTTPS

OBJECTIF

Déployer les services web et scheduler avec Portainer Repository, valider leur fonctionnement et publier le service en HTTPS.

MÉTHODE

Dépôt GitHub public, image GHCR et redéploiement depuis Portainer. Déploiement reproductible depuis le dépôt et le registre.

SOMMAIRE

1. Prérequis
2. Préparer le serveur cible
3. Déployer avec Portainer
4. Vérifier le déploiement
5. Configurer HTTPS
6. Mettre à jour
7. Dépannage
8. Checklist finale

1. Prérequis

- VM Linux avec une version Docker supportée par l'environnement Portainer.
- Portainer Community Edition, compte administrateur et accès SSH administrateur.
- Accès sortant vers GitHub et ghcr.io.
- Adresse IP fixe ; pour HTTPS, FQDN interne, certificat avec ce FQDN dans le SAN et CA approuvée par les clients.
- Trois chemins absolus pour les données, les documents et les certificats.
- PUID et PGID numériques, strictement supérieurs à zéro.
- Règles firewall limitées aux réseaux autorisés.

Élément	Valeur
Dépôt	https://github.com/Tetrax/upgrade_path_forti
Référence	refs/heads/main
Compose	docker-compose.portainer.yml
Image	ghcr.io/tetrax/upgrade_path_forti:latest
Services	web, scheduler
Listener interne	8000
Persistance	FORTIOS_DATA_DIR, FORTIOS_DOCS_DIR, FORTIOS_CERTS_DIR

NOTE

Utiliser la méthode Repository et l'image GHCR. Ne jamais stocker de clé privée ou de mot de passe dans Git, l'image ou une variable Portainer.

2. Préparer le serveur cible

Vérifier Docker, Portainer et les ports de la première installation :

```
docker version
docker ps --filter name=portainer
ss -tlnp | grep -E ':(8000|443)\b' || true
```

Docker et Portainer doivent répondre. 8000 doit être libre pour l'amorçage HTTP ; 443 doit être libre avant HTTPS.

Créer les chemins persistants. Adapter PUID, PGID et les trois chemins ensemble si la cible diffère :

```
PUID=1000
PGID=1000
DATA_DIR=/opt/upgrade_path/data
DOCS_DIR=/opt/upgrade_path/docs
CERTS_DIR=/opt/upgrade_path/certificates

mkdir -p "$DATA_DIR" "$DOCS_DIR" "$CERTS_DIR"
chown -R "$PUID:$PGID" "$DATA_DIR" "$DOCS_DIR"
```

Le conteneur applique au répertoire des certificats une politique distincte root:PGID. Ne pas lui appliquer un chown uniforme. Les trois chemins doivent être préservés lors de toute recreation.

3. Déployer avec Portainer

1. Ouvrir Stacks -> Add stack.
2. Nommer la stack upgrade-path.
3. Choisir Repository.
4. Renseigner :

```
Repository URL      : https://github.com/Tetrax/upgrade_path_forti
Repository reference : refs/heads/main
Compose path       : docker-compose.portainer.yml
```

1. Ajouter les variables suivantes, en remplaçant IP_LAN_VM et les chemins si nécessaire :

```
PUID=1000
PGID=1000
FORTIOS_HTTP_BIND_ADDRESS=0.0.0.0
FORTIOS_HTTP_PORT=8000
FORTIOS_TLS_CERT=
FORTIOS_TLS_KEY=
FORTIOS_TLS_HOSTNAME=
FORTIOS_RUN_ON_START=0
FORTIOS_EMAIL_ENABLED=false
FORTIOS_APP_URL=http://IP_LAN_VM:8000/app/
FORTIOS_DATA_DIR=/opt/upgrade_path/data
FORTIOS_DOCS_DIR=/opt/upgrade_path/docs
FORTIOS_CERTS_DIR=/opt/upgrade_path/certificates
```

1. Cliquer Deploy the stack.

Portainer tire ghcr.io/tetrax/upgrade_path_forti:latest et crée web et scheduler. Le bind 0.0.0.0 impose un filtrage firewall pendant l'amorçage.

4. Vérifier le déploiement

Dans un nouveau terminal, résoudre le conteneur web avant de l'utiliser :

```
WEB_CONTAINER="$(docker ps \
  --filter label=com.docker.compose.project=upgrade-path \
  --filter label=com.docker.compose.service=web \
  --format '{{.Names}}')"
test "$(printf '%s\n' "$WEB_CONTAINER" | sed '/^$/d' | wc -l)" -eq 1

docker logs --tail 50 "$WEB_CONTAINER"
docker inspect --format '{{.State.Health.Status}}' \
  "$WEB_CONTAINER"
curl --fail --silent --show-error \
  http://127.0.0.1:8000/app/ >/dev/null
```

web doit être running (healthy) et la requête HTTP doit réussir. Contrôler séparément dans Portainer que scheduler est running et que ses logs annoncent ses prochains traitements ; il n'a pas de healthcheck Compose. Vérifier aussi le montage des trois chemins persistants.

5. Configurer HTTPS

L'application utilise un seul listener interne 8000, en HTTP ou en HTTPS, jamais les deux. Le certificat doit contenir le FQDN dans le SAN.

Installer un PFX/P12

Chaque bloc est autonome et recalcule WEB_CONTAINER :

```
WEB_CONTAINER="$(docker ps \
  --filter label=com.docker.compose.project=upgrade-path \
  --filter label=com.docker.compose.service=web \
  --format '{{.Names}}')"
test "$(printf '%s\n' "$WEB_CONTAINER" | sed '/^$/d' | wc -l)" -eq 1

docker cp /chemin/certificat.pfx \
  "$WEB_CONTAINER":/tmp/certificat.pfx
read -rsp 'Mot de passe PFX : ' PFX_PASSWORD; echo
printf '%s' "$PFX_PASSWORD" | docker exec -i \
  "$WEB_CONTAINER" sh -c \
  'umask 077; cat > /tmp/cert-password'
unset PFX_PASSWORD

docker exec "$WEB_CONTAINER" fortios-certctl install \
  /tmp/certificat.pfx \
  --password-file /tmp/cert-password \
  --hostname upgrade-path.sns-security.lan \
  --output-dir /opt/fortios/certificates/active
docker exec "$WEB_CONTAINER" rm -f \
  /tmp/certificat.pfx /tmp/cert-password
```

Pour un PFX sans mot de passe, omettre le fichier de mot de passe et l'option correspondante. L'option --chain est interdite avec un PFX/P12.

Installer un certificat, une clé et une chaîne séparés

```
WEB_CONTAINER="$(docker ps \
  --filter label=com.docker.compose.project=upgrade-path \
  --filter label=com.docker.compose.service=web \
  --format '{{.Names}}')"
test "$(printf '%s\n' "$WEB_CONTAINER" | sed '/^$/d' | wc -l)" -eq 1

docker cp /chemin/serveur.crt "$WEB_CONTAINER":/tmp/serveur.crt
docker cp /chemin/serveur.key "$WEB_CONTAINER":/tmp/serveur.key
docker cp /chemin/chaine.p7b "$WEB_CONTAINER":/tmp/chaine.p7b
docker exec "$WEB_CONTAINER" fortios-certctl install \
  /tmp/serveur.crt \
  --key /tmp/serveur.key \
  --chain /tmp/chaine.p7b \
  --hostname upgrade-path.sns-security.lan \
  --output-dir /opt/fortios/certificates/active
docker exec "$WEB_CONTAINER" rm -f \
  /tmp/serveur.crt /tmp/serveur.key /tmp/chaine.p7b
```

Omettre la copie et l'option --chain si aucune chaîne séparée n'est fournie.

Activer HTTPS

Dans les variables de la stack, remplacer :

```
FORTIOS_HTTP_BIND_ADDRESS=0.0.0.0
FORTIOS_HTTP_PORT=443
FORTIOS_TLS_CERT=/opt/fortios/certificates/active/fullchain.pem
FORTIOS_TLS_KEY=/opt/fortios/certificates/active/privkey.pem
FORTIOS_TLS_HOSTNAME=upgrade-path.sns-security.lan
FORTIOS_APP_URL=https://upgrade-path.sns-security.lan/app/
```

Cliquer Update the stack. Le port hôte 443 pointe alors vers le listener interne 8000 devenu HTTPS ; le port hôte 8000 n'est plus publié. Après la recreation, vérifier avec un bloc autonome :

```

WEB_CONTAINER="$(docker ps \
  --filter label=com.docker.compose.project=upgrade-path \
  --filter label=com.docker.compose.service=web \
  --format '{{.Names}}')"
test "$(printf '%s\n' "$WEB_CONTAINER" | sed '/^$/d' | wc -l)" -eq 1

docker inspect --format '{{.State.Health.Status}}' \
  "$WEB_CONTAINER"
docker logs --tail 50 "$WEB_CONTAINER"
curl --fail --silent --show-error \
  https://upgrade-path.sns-security.lan/app/ >/dev/null

```

Le résultat doit être healthy et HTTPS doit répondre depuis un poste qui résout le FQDN et approuve la CA.

6. Mettre à jour

Le workflow publie latest et un tag immuable ghcr.io/tetrax/upgrade_path_forti:<SHA>. Avant toute mise à jour, relever un <SHA_PRECEDENT> connu et sauvegarder à froid les trois chemins réellement configurés.

1. Arrêter la stack depuis Portainer, puis exécuter :

```

DATA_DIR=/opt/upgrade_path/data
DOCS_DIR=/opt/upgrade_path/docs
CERTS_DIR=/opt/upgrade_path/certificates
BACKUP_DIR="/srv/backups/upgrade-path/$(date +%Y%m%d-%H%M%S)"

install -d -m 700 "$BACKUP_DIR"
cp -a "$DATA_DIR" "$BACKUP_DIR/data"
cp -a "$DOCS_DIR" "$BACKUP_DIR/docs"
cp -a "$CERTS_DIR" "$BACKUP_DIR/certificates"

```

1. Redémarrer la stack, ouvrir upgrade-path, cliquer Pull and redeploy puis confirmer avec Update.
2. Après la recreation, recalculer WEB_CONTAINER dans le même bloc qui le contrôle :

```

WEB_CONTAINER="$(docker ps \
  --filter label=com.docker.compose.project=upgrade-path \
  --filter label=com.docker.compose.service=web \
  --format '{{.Names}}')"
test "$(printf '%s\n' "$WEB_CONTAINER" | sed '/^$/d' | wc -l)" -eq 1

docker inspect --format '{{.State.Health.Status}}' \
  "$WEB_CONTAINER"
docker logs --tail 50 "$WEB_CONTAINER"

```

web doit être healthy. Dans Portainer, scheduler doit être running et ses logs doivent rester cohérents. Tester HTTPS et vérifier les trois montages.

Pour revenir en arrière, faire valider dans la référence Git utilisée par la stack un compose où les deux lignes image: pointent vers ghcr.io/tetrax/upgrade_path_forti:<SHA_PRECEDENT>, puis refaire Pull and redeploy -> Update. latest ne constitue pas un rollback fiable.

Si les données doivent aussi être restaurées, arrêter la stack, conserver les chemins en échec, puis restaurer la sauvegarde :

```

DATA_DIR=/opt/upgrade_path/data
DOCS_DIR=/opt/upgrade_path/docs
CERTS_DIR=/opt/upgrade_path/certificates
BACKUP_DIR='/srv/backups/upgrade-path/<SAUVEGARDE_VALIDÉE>'
FAILED_SUFFIX="failed-$(date +%Y%m%d-%H%M%S) "

mv "$DATA_DIR" "${DATA_DIR}.${FAILED_SUFFIX}"
mv "$DOCS_DIR" "${DOCS_DIR}.${FAILED_SUFFIX}"
mv "$CERTS_DIR" "${CERTS_DIR}.${FAILED_SUFFIX}"
cp -a "$BACKUP_DIR/data" "$DATA_DIR"
cp -a "$BACKUP_DIR/docs" "$DOCS_DIR"
cp -a "$BACKUP_DIR/certificates" "$CERTS_DIR"

```

Redémarrer ensuite la stack épinglée au SHA précédent et refaire les contrôles des chapitres 4 et 5. Tester cette restauration hors production avant de l'adopter.

7. Dépannage

Symptôme	Vérification ou action
Stack non déployée	Vérifier URL, référence, compose, variables de chemins et accès à ghcr.io.
web non healthy	Recalculer son nom, lire ses logs et vérifier que les variables TLS sont fournies ensemble.
scheduler arrêté	Lire ses logs et contrôler les montages data/docs et ses variables.
Certificat refusé	Vérifier SAN, dates, usage serveur, clé correspondante et chaîne.
Alerte navigateur	Vérifier DNS, SAN, chaîne et confiance dans la CA.
HTTP encore exposé	Confirmer le port hôte 443, mettre à jour la stack et vérifier le firewall.
Données absentes	Vérifier les trois chemins et leurs montages ; ne rien supprimer avant diagnostic.
Mise à jour défaillante	Épingler le SHA précédent ; restaurer les chemins seulement si nécessaire et depuis une sauvegarde testée.

8. Checklist finale

- [] Docker et Portainer sont opérationnels.
- [] Les trois chemins persistants existent avec les droits adaptés à PUID et PGID.
- [] La stack utilise Repository, la bonne référence et le bon compose.
- [] web est healthy ; scheduler est running et ses logs sont cohérents.
- [] WEB_CONTAINER est recalculé dans chaque nouveau terminal et après chaque recreation.
- [] HTTPS répond avec le bon FQDN, le bon SAN et une CA approuvée.
- [] Le firewall n'expose que le port attendu aux réseaux autorisés.
- [] Une sauvegarde à froid des trois chemins et sa restauration ont été testées.
- [] Le SHA précédent est connu et la procédure Pull and redeploy -> Update est validée.